

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Thaís dos Reis Soares
Matrícula: 825322

Implementando Padrões de Teste (Test Patterns)

Disciplina: Teste de Software

1. Padrões de criação de dados (*builders*)

Durante o desenvolvimento da suíte de testes, foram usados dois padrões de criação de dados: o *Object Mother* e o *Data Builder*.

1.1 Por que o *CarrinhoBuilder* em vez de um *CarrinhoMother*?

O padrão *Object Mother* foi usado para a criação de objetos do tipo *User*, pois este é um objeto simples, com poucos estados relevantes para os testes. No projeto, existem apenas dois tipos de usuário, o padrão e o *premium*, por isso a classe *UserMother* é suficiente para resolver o problema da criação de dados para esta classe.

Já o padrão *Data Builder* foi usado para a criação de instâncias do *Carrinho*, pois este é um objeto mais complexo. Um carrinho pode ter dois tipos de usuário, o padrão ou o *premium*, vários itens ou estar vazio. Usar o padrão *Object Mother* nesse caso poderia gerar vários métodos, um para cada possível variação do carrinho. O padrão *Data Builder* resolve esse problema ao permitir a construção do objeto de forma flexível, por meio de métodos que modificam apenas os atributos relevantes para cada cenário, mantendo valores padrão para os demais. Isso torna os testes mais claros, reutilizáveis e fáceis de manter.

1.2 “Antes” e “depois” do *Data Builder*

A seguir, serão apresentados dois cenários: o “antes”, que representa a criação de objetos sem o uso do padrão *Data Builder*, e o “depois”, que mostra a criação dos objetos utilizando este padrão.

```
const user = new User('2', 'Miranda Vieira', 'miranda@gmail.com', 'PREMIUM');  
const itens = [new Item('Batata', 200)];  
const carrinho = new Carrinho(user, itens);
```

Figura 1 - “antes”

A figura 1 mostra a criação dos objetos sem o *Data Builder*. Conforme ilustrado por ela, os objetos devem ser instanciados manualmente dentro do teste. Essa abordagem faz com que o código se torne extenso e repetitivo, especialmente quando diferentes cenários precisam ser testados.

```
const carrinho = new CarrinhoBuilder()  
    .comUser(user)  
    .comItens([new Item('Batata', 200)])  
    .build();
```

Figura 2 - “depois”

A figura 2 mostra a criação dos objetos utilizando o padrão *Data Builder*. Conforme observado, apenas os dados relevantes para os testes são manipulados, enquanto os demais utilizam valores padrão definidos pelo *Builder*. Isso reduz o tamanho do código de teste, tornando-o mais legível e fácil de compreender.

1.3 Como o *Builder* melhora a legibilidade e a manutenção

Como foi apresentado nas figuras 1 e 2, o padrão *Data Builder* simplifica a criação de objetos utilizados nos testes. Em vez de instanciar manualmente diversos objetos e repetir o mesmo código em diferentes cenários, ele permite modificar apenas os atributos relevantes para cada teste, mantendo valores padrão para os demais. Isso promove a reutilização de código e diminui a duplicação da lógica de criação de objetos, facilitando a manutenção da suíte de testes.

Além disso, essa abordagem reduz a quantidade de código necessária para o *setup* dos testes, tornando-os mais limpos, legíveis e fáceis de compreender. Assim, o foco do teste passa a ser o comportamento que está sendo validado, e não os detalhes da criação dos objetos necessários para sua execução.

2. Padrões de *Test Doubles* (*Mocks* vs. *Stubs*)

A análise a seguir é baseada no teste da Etapa 5: o cenário em que um cliente *premium* finaliza a compra com sucesso, aplicando 10% de desconto e recebendo um *e-mail* de confirmação.

2.1 Identificando *Stub* e *Mock*

Neste teste, foram utilizados diferentes tipos de *Test Doubles* para isolar o sistema sob teste das dependências externas. O método testado foi o *processarPedido*, que usa três dependências externas: o *GatewayPagamento*, o *PedidoRepository* e o *EmailService*.

O *GatewayPagamento* foi utilizado como *stub*, pois o objetivo era fornecer uma resposta "enlatada" que permitisse o fluxo de execução avançar. Neste caso, a resposta esperada era *“true”*, pois o objetivo era testar o caminho de sucesso. O foco não estava em verificar como o *gateway* se comporta, mas sim em fornecer um

retorno previsível para o sistema. Da mesma forma, o `PedidoRepository` foi utilizado como *Stub*, retornando um pedido previamente configurado para permitir a continuidade do fluxo. Não é o objetivo do teste verificar se ou como ele foi chamado, de modo que ele só precisa fornecer os dados corretos quando necessário.

Já o `EmailService` foi utilizado como *Mock*, pois o objetivo do teste é verificar se o serviço foi chamado corretamente. O método não retorna o *e-mail*, ele o dispara como consequência de uma compra bem-sucedida. Não há como verificar esse comportamento olhando apenas para o valor de retorno do método. A única forma de confirmar que o serviço se comportou corretamente é perguntar ao dublê se ele foi chamado, quantas vezes e com quais argumentos, como mostra a figura 3. Isso é verificação de comportamento.

```
expect(emailMock.enviarEmail).toHaveBeenCalledTimes(1);
expect(emailMock.enviarEmail).toHaveBeenCalledWith(
  user.email,
  'Seu Pedido foi Aprovado!',
  `Pedido ${pedidoSalvo.id} no valor de R${pedidoSalvo.totalFinal}`
);
```

Figura 3

2.2 Mock vs. Stub

A diferença entre *Stubs* e *Mocks* está no tipo de verificação realizada pelo teste. Os *Stubs* foram utilizados para realizar a Verificação de Estado, fornecendo respostas controladas para que fosse possível validar o resultado produzido pelo método `processarPedido`. Já o *Mock* foi utilizado para realizar a Verificação de Comportamento, permitindo verificar se o método `enviarEmail` foi chamado corretamente, com os parâmetros esperados. Dessa forma, os *Stubs* controlam o fluxo de execução, enquanto o *Mock* valida as interações entre o sistema e suas dependências.

3. Conclusão

O desenvolvimento desta suíte de testes tornou evidente como o uso deliberado de Padrões de Teste ajuda a prevenir contra *Test Smells*.

O padrão *Data Builder* elimina o *smell* de *Obscure Setup*. Ao centralizar a lógica de construção de objetos e oferecer uma API fluente para customização, o *Builder* garante que cada teste declare apenas o que é relevante para o seu cenário, tornando a intenção do teste imediatamente legível e reduzindo o custo de manutenção.

Os padrões de *Test Doubles* eliminam os *smells* de *Fragile Tests* e *Slow Tests* causados por dependências externas reais. Ao substituir *gateways* de pagamento, repositórios e serviços de *e-mail* por dublês controlados, os testes tornam-se rápidos, determinísticos e isolados, podendo ser executados a qualquer momento, sem dependência externa.

Portanto, o uso deliberado desses padrões melhora a organização e a legibilidade dos testes, atuando de forma preventiva contra *Test Smells*. Isso contribui para a construção de uma suíte de testes mais sustentável, confiável e fácil de manter ao longo da evolução do sistema.

Link do repositório do projeto: [thais-reiss/test-pattern](https://github.com/thais-reiss/test-pattern)